

PAY-68

1. Number of Islands:

a. Traverse every cell in grid

b. when you find a '1', that means you've discovered a new island.

c. Use dfs to mark all connected lands cells as visited (turn to '0')

d. Increment island Count.

def numIslands(grid):

if not grid:

return 0

rows, cols = len(grid), len(grid[0])

count = 0

def dfs(r, c):

if r < 0 or r > rows or c < 0 or c > cols or

grid[r][c] == '0':

return

grid[r][c] = '0'

dfs(r+1, c)

dfs(r-1, c)

dfs(r, c+1)

dfs(r, c-1)

for r in range(rows):

for c in range(cols):

if grid[r][c] == '1':

count += 1

dfs(r, c)

return count

grid = [[1, 1, 1, 1, 0],
[1, 1, 0, 1, 0]]

dfs(0,0)
visited visited
= [[0, 0, 0, 0, 0],
[0, 0, 0, 0, 0]]

2. Max Area of Island

return max

instead of counting island, compute size of each island

```
def maxArea(grid):
```

```
    if not grid:
```

```
        return 0
```

```
    rows, cols = len(grid), len(grid[0])
```

```
    maxArea = 0
```

```
    def dfs(r, c):
```

grid[r][c] == 0:

```
        if r < 0 or r > rows or c < 0 or c > cols or
```

```
            return 0
```

```
        grid[r][c] = 0
```

```
        area = 1 # Current cell contributes 1 + neighbours
```

```
        area += dfs(r+1, c)
```

```
        area += dfs(r-1, c)
```

```
        area += dfs(r, c+1)
```

```
        area += dfs(r, c-1)
```

```
        return area
```

```
    for r in range(rows):
```

```
        for c in range(cols):
```

```
            if grid[r][c] == 1:
```

```
                maxArea = max(maxArea, dfs(r, c))
```

```
    return maxArea
```

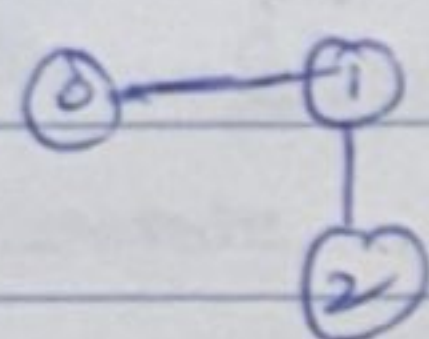
TC = $O(m \times n)$

SC = $O(m \times n)$ worst case (recursion stack)

3. Number of Connected Components in an undirected Graph:-

This is graph version of "No. of islands".

$n=5$, edges = $[[0,1],[1,2],[3,4]]$ o/p: - 2



each node = cell
each component = island

```
def CountComponents(n, edges):
```

```
    graph = {i: [] for i in range(n)}
```

```
    for u, v in edges:
```

```
        graph[u].append(v)
```

```
        graph[v].append(u)
```

```
    visited = set()
```

```
    def dfs(node):
```

```
        visited.add(node)
```

```
        for nei in graph[node]: // visit all neighbours
```

```
            if nei not in visited:
```

```
                dfs(nei)
```

```
    components = 0
```

```
    for i in range(n):
```

```
        if i not in visited:
```

```
            components += 1 # New Component found
```

```
            dfs(i)
```

```
    return components
```


graph: $\{ 0 \rightarrow [1], 1 \rightarrow [0, 2], 2 \rightarrow [1], 3 \rightarrow [4], 4 \rightarrow [3] \}$

Steps	Current Node	Action	Visited Set	Components Count
1	0	Not visited $\rightarrow$ start dfs(0)	{0}	1
2	0 $\rightarrow$ 1	visit 1	{0, 1}	1
3	1 $\rightarrow$ 2	visit 2	{0, 1, 2}	1
4	2	no new neighbours	{0, 1, 2}	1
5	1	already processed	{0, 1, 2}	1
6	3	not visited $\rightarrow$ start dfs(3)	{0, 1, 2, 3}	2
7	3 $\rightarrow$ 4	visit 4	{0, 1, 2, 3, 4}	2
8	4	no new neighbours	{0, 1, 2, 3, 4}	2
9	4	done	{0, 1, 2, 3, 4}	² (2) ✓